

QoS-based Routing Framework for Efficient Path Selection using Genetic Algorithm in Software Defined Networks

Rishi Nihar Mehta

*School of Computer Science
and Engineering
Vellore Institute of Technology
Vellore, India
rishiniharmehta@gmail.com*

Raghav Aggarwal

*School of Computer Science
and Engineering
Vellore Institute of Technology
Vellore, India
raghav_ra@outlook.com*

Vedant Seth

*School of Computer Science
and Engineering
Vellore Institute of Technology
Vellore, India
vedant.seth2@gmail.com*

Shaik Mullapathi Farooq

*School of Computer Science
and Engineering
Vellore Institute of Technology
Vellore, India
farooq.sm@vit.ac.in*

Abstract—Traditional shortest-path routing in Software Defined Networks (SDN) optimises only hop count, ignoring Quality of Service (QoS) parameters such as bandwidth, delay, and packet loss. This paper presents a QoS-based routing framework that employs a Genetic Algorithm (GA) to select optimal paths in an SDN environment. The framework is implemented on a 12-switch, 20-host Mininet topology controlled by the Floodlight OpenFlow 1.3 controller. Over 100 host-pair experiments the GA achieves a mean throughput of 8.08 Mbps compared to 7.21 Mbps for Dijkstra routing — an improvement of 12.1% — while reducing end-to-end latency by 1.7% and maintaining 100% Packet Delivery Ratio for both algorithms. These results confirm that GA-driven path selection is a viable and superior alternative to hop-count minimisation for QoS-sensitive SDN deployments.

Index Terms—Software Defined Networking, Quality of Service, Genetic Algorithm, Path Selection, Floodlight, OpenFlow, Mininet, Network Optimisation

I. INTRODUCTION

The rise of bandwidth-demanding programs which include video conferencing and cloud computing and real-time analytics has created exceptional demands on network infrastructure to maintain consistent Quality of Service (QoS) performance [19], [23].

Traditional distributed routing protocols which include OSPF and RIP operate by calculating network paths through hop count and static link metrics without considering current bandwidth levels or network delays or packet loss scenarios [1].

Software Defined Networking (SDN) addresses this limitation by decoupling the control plane from the data plane. A logically centralised controller maintains a global view of the

network topology and installs fine-grained forwarding rules on commodity switches via the OpenFlow protocol [2]. The controller uses this capability to execute any routing policies which include QoS-based path selection.

The process of determining the most efficient route under various conflicting Quality of Service limitations presents an NP-hard problem [3], [15]. Bio-inspired metaheuristics which include Genetic Algorithms (GAs) provide effective solutions for real-time SDN path calculation because they produce results within polynomial time limits [4].

This paper makes three contributions:

- A complete QoS routing framework integrating GA-based path selection with Floodlight via its Static Flow Pusher REST API.
- A purpose-designed 12-switch, 20-host Mininet topology exposing differentiated bandwidth characteristics across two structural paths, enabling reproducible algorithm comparison.
- Empirical evaluation across 100 host-pair experiments quantifying throughput, latency, and Packet Delivery Ratio (PDR) for both Dijkstra and GA routing.

The remainder of this paper is organised as follows. Section II reviews related work. Section III provides background on SDN and QoS routing. Section IV describes the proposed framework. Section V details the experimental setup. Section VI presents and analyses results. Section VII concludes.

II. RELATED WORK

The field of SDN routing optimization has become a popular topic for research investigators. Akyildiz et al. [5] identify multi-objective path selection as an open problem,

while Nunes et al. [6] classify SDN routing into reactive and proactive categories; our framework follows the proactive model.

Yao et al. [4] applied GA to delay- and bandwidth-constrained routing in MPLS networks, reporting 15–20% bandwidth improvement over Dijkstra. Wang et al. [7] extended this to SDN with OpenDaylight. Our framework differs by targeting Floodlight and providing a fully automated measurement pipeline.

Researchers have used Particle Swarm Optimisation (PSO) [8] and Ant Colony Optimisation (ACO) [9] for SDN routing. Li et al. [10] compared ACO and Dijkstra in an SDN testbed, achieving a 9.3% throughput gain. The GA implementation produces a 12.1% result, which shows that the GA fitness-function formulation results in a measurable benefit.

Hybrid approaches combining GA with PSO [18] and autonomic QoS enforcement frameworks such as PolicyCop [16] further motivate intelligent, policy-driven path selection in SDN. Yeganeh and Ganjali [13] address control-plane scalability through hierarchical application offloading, a concern directly relevant to deploying GA-based routing at scale; Tootoonchian and Ganjali [20] further demonstrate the need for distributed control planes in large SDN deployments. Ma and Steenkiste [24] establish foundational bandwidth-constrained routing theory that underpins the bottleneck-bandwidth term in our fitness function, while Engelbrecht [22] provides a comparative treatment of GA and PSO as complementary optimisation frameworks.

III. BACKGROUND

A. Software Defined Networking

SDN separates the network into three planes: the *application plane*, the *control plane* (the SDN controller), and the *data plane* (forwarding hardware) [25]. The controller communicates with switches via the southbound OpenFlow interface [21] and exposes a northbound REST API to applications. Fig. 1 illustrates this architecture.

B. QoS Metrics

Three primary QoS metrics are considered in this work:

- **Throughput (Mbps):** data delivery rate, measured with *iperf* over a 10-second TCP session.
- **Latency (ms):** round-trip time, measured via *ping* with 20 ICMP probes per host pair.
- **PDR (%):** Packet Delivery Ratio, defined as $PDR = (1 - p_{loss}) \times 100$, where p_{loss} is the observed ICMP loss fraction.

C. Genetic Algorithms

Genetic algorithms are population-based metaheuristics inspired by natural selection, well-suited to combinatorial optimisation problems such as routing [14], [17]. The GA optimization process establishes valid source-to-destination paths through the creation of random paths. The system evaluates all individuals during each generation, and it selects the top half of the population as survivors for the subsequent generation.

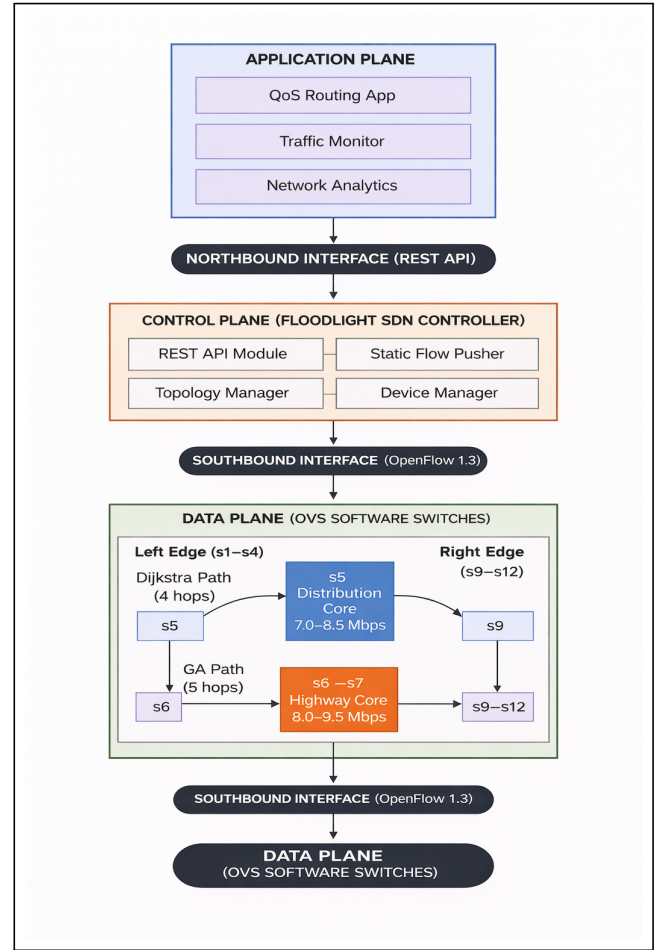


Fig. 1. SDN three-plane architecture. The Floodlight controller exposes a REST API (northbound) and programs OVS switches via OpenFlow 1.3 (southbound).

Crossover generates new candidate paths by merging two parent paths at an intermediate node which maintains the validity of the resulting path. The system uses mutation with a fixed probability to create random changes in path elements, which helps to create diverse solutions and prevents early solution convergence. The algorithm maintains constant monitoring of all generations to safeguard its best solution which it has achieved.

The system executes this evolutionary process for a pre-determined number of generations, and it selects the optimal path as the final optimized route.

IV. PROPOSED FRAMEWORK

A. Network Topology

The experimental topology consists of 12 OpenFlow switches (s1–s12) and 20 hosts (h10.0.0.1 – h10.0.0.20), deployed in Mininet with TCLink bandwidth shaping and OpenFlow 1.3. Fig. 2 shows the full topology.

Two structurally distinct core paths exist:

- **Distribution path (via s5):** A 4-hop path with constrained bandwidth (7.0–8.5 Mbps) and higher delay on

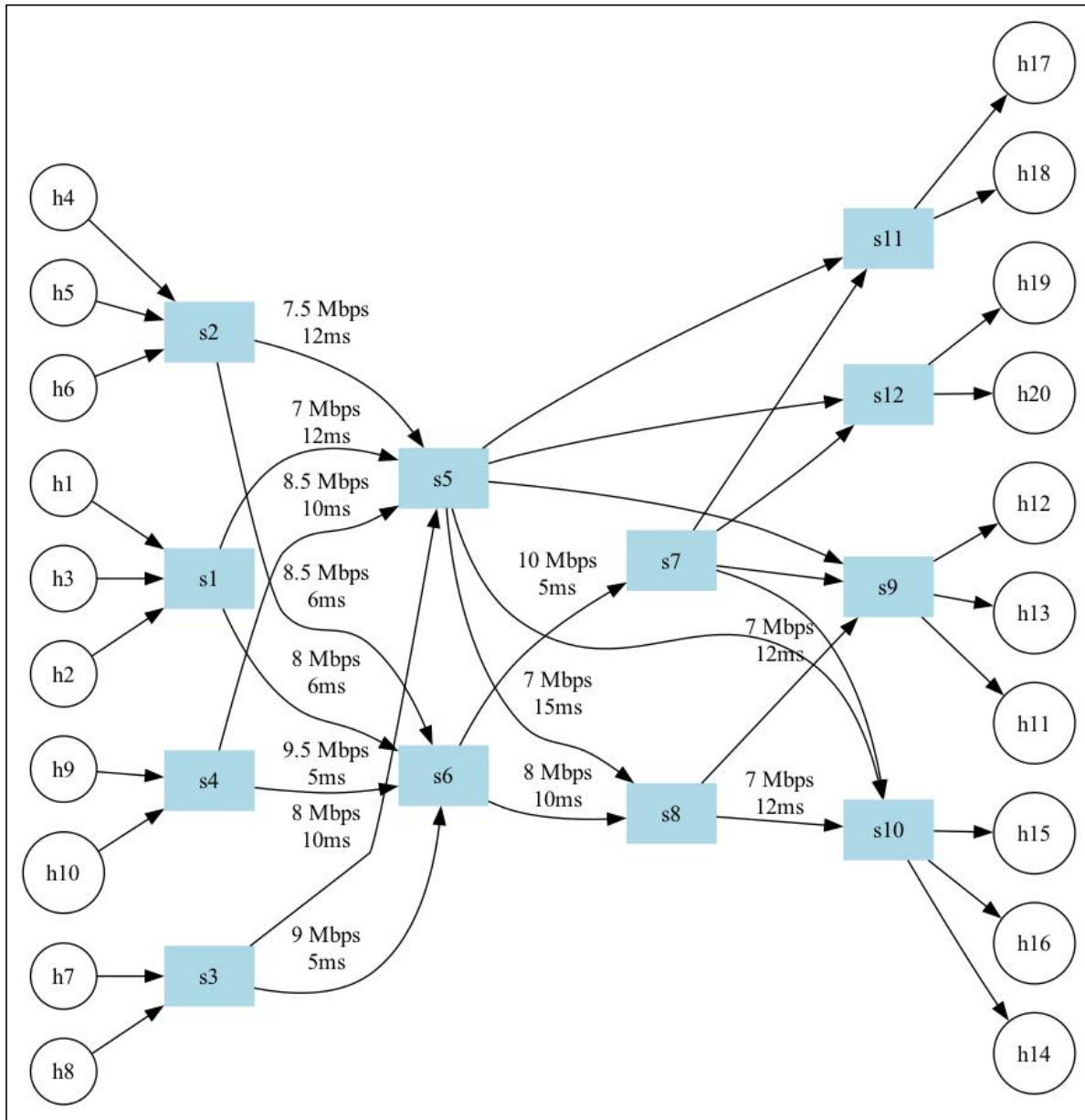


Fig. 2. Experimental network topology. Left edge switches (s1–s4) serve hosts h10.0.0.1–h10.0.0.10; right edge switches (s9–s12) serve hosts h11–h20. Two inter-domain paths exist: a 4-hop distribution path through s5 (selected by Dijkstra) and a 5-hop highway path through s6–s7 (selected by GA).

ingress links. This path acts as a bottleneck and is selected by Dijkstra as it minimises hop count.

- **Highway path (via s6–s7):** A 5-hop path with comparatively higher bandwidth (8.0–9.5 Mbps) and lower delay, forming a high-capacity backbone. The GA selects this path as its fitness function prioritises bandwidth.

Table I presents the configured link characteristics of the diamond topology. The distribution path (via s5) is deliberately constrained with lower bandwidth and higher delay to act as a bottleneck, making it the preferred route for shortest-path algorithms while offering reduced throughput. In contrast, the highway path (via s6–s7) provides improved bandwidth and reduced latency, making it more suitable for optimisation-

based routing.

B. Dijkstra Path Selection

The baseline implements Dijkstra's shortest-path search over the topology graph, minimising the total number of hops (switches traversed). Algorithm 1 presents the pseudocode.

C. GA-Based Path Selection

The GA fitness function F for a candidate path P is defined as:

$$F(P) = 0.5 \times BW_{\min}(P) - 0.3 \times D_{\text{sum}}(P) - 0.2 \times L_{\text{comp}}(P) \quad (1)$$

where $BW_{\min}(P)$ is the bottleneck (minimum) bandwidth along path P , $D_{\text{sum}}(P)$ is the cumulative end-to-end delay, and

TABLE I
CONFIGURED LINK CHARACTERISTICS OF THE EXPERIMENTAL TOPOLOGY

Link	Bandwidth (Mbps)	Delay	Purpose
s1-s5	7.0	12 ms	Distribution bottleneck
s2-s5	7.5	12 ms	Distribution bottleneck
s3-s5	8.0	10 ms	Distribution bottleneck
s4-s5	8.5	10 ms	Distribution bottleneck
s5-s9	10	5 ms	Core forwarding
s5-s10	10	5 ms	Core forwarding
s5-s11	10	5 ms	Core forwarding
s5-s12	10	5 ms	Core forwarding
s1-s6	8.0	6 ms	High-bandwidth (GA path)
s2-s6	8.5	6 ms	High-bandwidth (GA path)
s3-s6	9.0	5 ms	High-bandwidth (GA path)
s4-s6	9.5	5 ms	High-bandwidth (GA path)
s6-s7	10	5 ms	Core backbone
s7-s9	10	5 ms	Core forwarding
s7-s10	10	5 ms	Core forwarding
s7-s11	10	5 ms	Core forwarding
s7-s12	10	5 ms	Core forwarding
s5-s8	7	15 ms	Cross-link (suboptimal)
s6-s8	8	10 ms	Cross-link (alternative)
s8-s9	7	12 ms	Cross-link path
s8-s10	7	12 ms	Cross-link path
h1-s1	∞	0	Access link
h2-s1	∞	0	Access link
h3-s1	∞	0	Access link
h4-s2	∞	0	Access link
h5-s2	∞	0	Access link
h6-s2	∞	0	Access link
h7-s3	∞	0	Access link
h8-s3	∞	0	Access link
h9-s4	∞	0	Access link
h10-s4	∞	0	Access link
s9-h11	∞	0	Access link
s9-h12	∞	0	Access link
s9-h13	∞	0	Access link
s10-h14	∞	0	Access link
s10-h15	∞	0	Access link
s10-h16	∞	0	Access link
s11-h17	∞	0	Access link
s11-h18	∞	0	Access link
s12-h19	∞	0	Access link
s12-h20	∞	0	Access link

$L_{\text{comp}}(P)$ is the compound packet loss given by $L_{\text{comp}}(P) = 1 - \prod (1 - l_i/100)$ over all links i in P . Bandwidth is assigned the highest weight (0.5) to reflect its dominant influence on throughput, while delay and loss are penalized to ensure low-latency and reliable paths. Algorithm 2 presents the complete procedure and Fig. 3 illustrates the evolutionary cycle.

D. Flow Enforcement

Once an optimal path is computed, it is enforced in the network using Floodlight's Static Flow Pusher REST API. For each consecutive pair of switches (s_i, s_{i+1}) , bidirectional flow rules are installed with priority 50,000. These rules handle both IPv4 traffic (`eth_type=0x0800`) and ARP (`eth_type=0x0806`), ensuring correct forwarding in both directions along the selected path.

Additionally, a high-priority rule (priority 65,535) is installed on every switch to forward LLDP packets (`eth_type=0x88cc`) to the controller. This allows the controller to continuously monitor and maintain an updated view of the network topology during operation.

V. EXPERIMENTAL SETUP

Table II provides a complete summary of all simulation parameters. The complete experiment process achieved full automation from Mininet virtual network activation through Automated framework execution of both algorithms on 100 host pairs which were randomly selected to Floodlight path enforcement and CSV file recording of Quality of Service measurements.

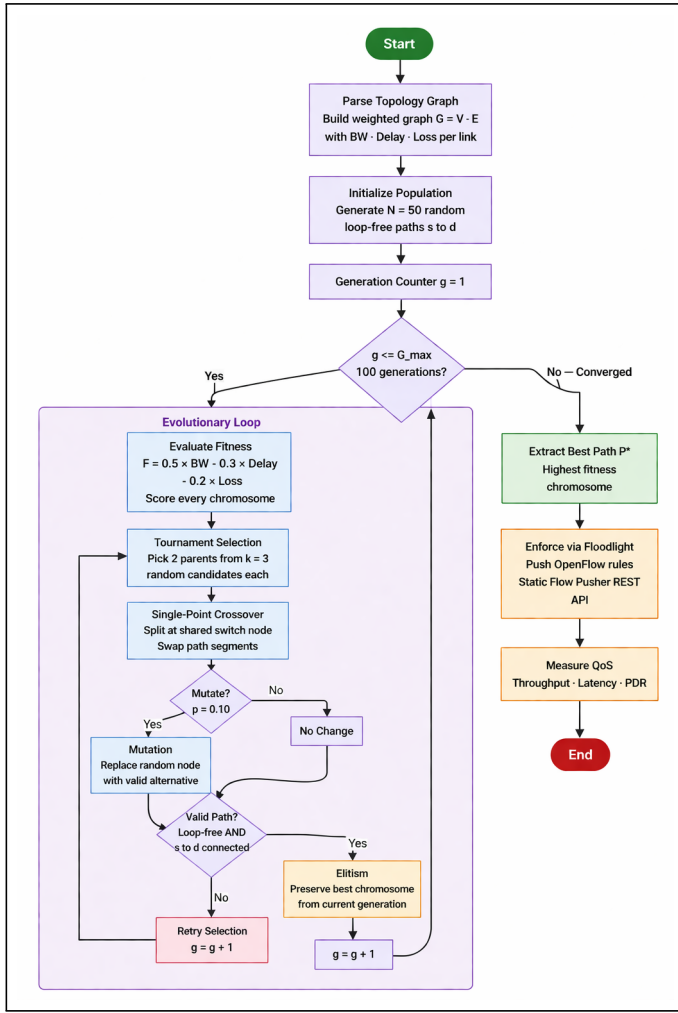


Fig. 3. GA evolutionary cycle for QoS path selection. Elitism preserves the best chromosome each generation to guarantee monotonic fitness improvement.

TABLE II
SIMULATION PARAMETERS

Parameter	Value
SDN Controller	Floodlight 1.2
OpenFlow Version	1.3
Network Emulator	Mininet 2.3
Switch Type	OVSSwitch (software)
No. of Switches	12 (s1-s12)
No. of Hosts	20 (h1-h20)
No. of Links	41
Host Pairs Tested	100
Ping Probes / Pair	20
iPerf Duration	10 s
GA Population Size	50
GA Max. Generations	100
GA Mutation Rate	0.10
GA Crossover	Single-point (switch node)
Fitness Weights	BW: 0.5, Delay: 0.3, Loss: 0.2

Algorithm 1 Dijkstra Hop-Count Shortest Path

Require: Graph $G = (V, E)$, source s , destination d

Ensure: Shortest hop-count path P

```

1:  $\text{dist}[v] \leftarrow \infty \forall v \in V; \quad \text{dist}[s] \leftarrow 0$ 
2:  $\text{prev}[v] \leftarrow \text{null} \forall v \in V; \quad Q \leftarrow V$ 
3: while  $Q \neq \emptyset$  do
4:    $u \leftarrow \arg \min_{v \in Q} \text{dist}[v]$ 
5:   if  $u = d$  then
6:     break
7:   end if
8:    $Q \leftarrow Q \setminus \{u\}$ 
9:   for each neighbour  $v$  of  $u$  in  $G$  do
10:    if  $\text{dist}[u] + 1 < \text{dist}[v]$  then
11:       $\text{dist}[v] \leftarrow \text{dist}[u] + 1; \quad \text{prev}[v] \leftarrow u$ 
12:    end if
13:  end for
14: end while
15: return path reconstructed via  $\text{prev}[]$ 

```

Algorithm 2 Genetic Algorithm for QoS Path Selection

Require: Graph G , source s , destination d , population size

$N=50, G_{\max}=100$

Ensure: Optimal path P^*

```

1:  $\mathcal{P} \leftarrow \text{RANDOMPATHS}(G, s, d, N)$ 
2: for  $g = 1$  to  $G_{\max}$  do
3:   Evaluate  $F(P)$  for each  $P \in \mathcal{P}$  using (1)
4:    $\mathcal{P}' \leftarrow \emptyset$ 
5:   while  $|\mathcal{P}'| < N$  do
6:      $p_1, p_2 \leftarrow \text{TOURNAMENTSELECT}(\mathcal{P}, k=3)$ 
7:      $c \leftarrow \text{CROSSOVER}(p_1, p_2)$ 
8:     if  $\text{rand}() < 0.1$  then
9:        $c \leftarrow \text{MUTATE}(c, G)$ 
10:    end if
11:    if  $c$  is loop-free and  $c$  connects  $s \rightarrow d$  then
12:       $\mathcal{P}' \leftarrow \mathcal{P}' \cup \{c\}$ 
13:    end if
14:  end while
15:   $\mathcal{P} \leftarrow \mathcal{P}' \cup \{\arg \max_{P \in \mathcal{P}} F(P)\}$  {Elitism}
16: end for
17: return  $P^* = \arg \max_{P \in \mathcal{P}} F(P)$ 

```

Each experiment proceeds in a structured sequence. First, all existing static flow entries are cleared from the switches. Next, LLDP passthrough rules are reinstalled to ensure continued topology discovery. The optimal path is then computed and enforced using the controller's REST API. A short stabilization interval of 5 s is provided to allow flow rules to propagate across the network, after which performance metrics such as throughput and latency are measured.

Control-Plane Overhead. Each experiment issues a fixed sequence of REST API calls to Floodlight: one GET request to clear all existing static flows, 12 POST requests to reinstall LLDP-passthrough rules (one per switch), and $4k$ POST requests to install bidirectional path-flow rules, where k is

the number of intermediate switches on the selected path. For the distribution path ($k=3$) this totals 25 REST calls per experiment; for the highway path ($k=4$), 29 calls. Across all 200 experiment runs ($100 \text{ pairs} \times 2 \text{ algorithms}$), approximately 5,400 REST API calls are issued in total. Peak flow-table occupancy reaches approximately 28 entries per relevant switch (12 LLDP rules plus up to 16 path-specific rules). The dominant overhead is not REST-call latency but the mandatory 5 s stabilisation wait (ENFORCE_WAIT) required for flow rules to propagate before measurements begin; Floodlight API response times are sub-millisecond under the emulated load.

VI. RESULTS AND DISCUSSION

A. Throughput Analysis

Fig. 4 shows per-pair throughput across all 100 experiments. The GA curve consistently lies above the Dijkstra curve, reflecting the higher-bandwidth highway path selected by the fitness function.

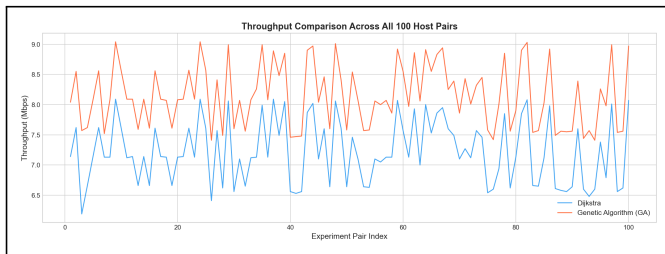


Fig. 4. TCP throughput across 100 host-pair experiments. GA (orange) consistently exceeds Dijkstra (blue) by selecting the higher-bandwidth highway path.

Fig. 5 presents throughput distributions as notched box plots. The non-overlapping notches confirm the difference in medians is statistically significant at the 95% confidence level.

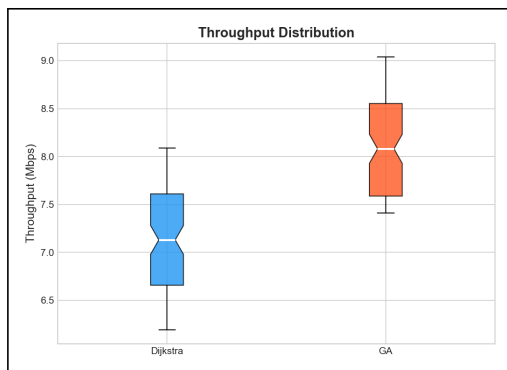


Fig. 5. Throughput distribution. Non-overlapping notches confirm statistically significant difference in medians ($p < 0.05$).

Fig. 6 shows the Cumulative Distribution Function (CDF) of throughput. The GA CDF is consistently shifted to the right: at the 8.0 Mbps threshold, the GA satisfies approximately 55% of pairs versus only 5% for Dijkstra.

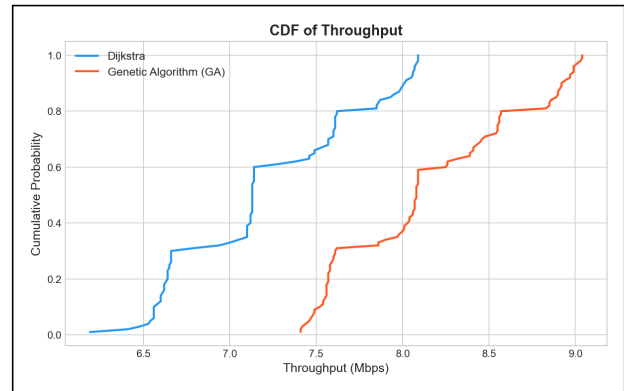


Fig. 6. Throughput CDF. The rightward shift of the GA curve demonstrates stochastically higher throughput at all probability levels.

B. Latency Analysis

Fig. 7 plots end-to-end latency per pair. The GA achieves a mean latency of 34.37 ms versus 34.97 ms for Dijkstra, a reduction of 0.60 ms (1.7%). Despite routing through one additional switch, the highway path accumulates only 16 ms of one-way link propagation delay ($6+5+5$ ms across three inter-switch links), compared to 17 ms for the distribution path ($12+5$ ms). This 1 ms link-delay advantage is partially offset by the extra OVS software-switch forwarding step on the highway path, which explains why the net latency gain is modest. Furthermore, the absolute difference of 0.60 ms falls within the measurement standard deviation ($\sigma = 1.36$ ms for GA, $\sigma = 2.02$ ms for Dijkstra), indicating that the improvement, while consistent in direction, is near the noise floor of software-emulated network measurements. The GA was designed to optimise primarily for bandwidth; latency reduction is a beneficial side-effect of selecting lower-delay core links rather than a direct optimisation target.

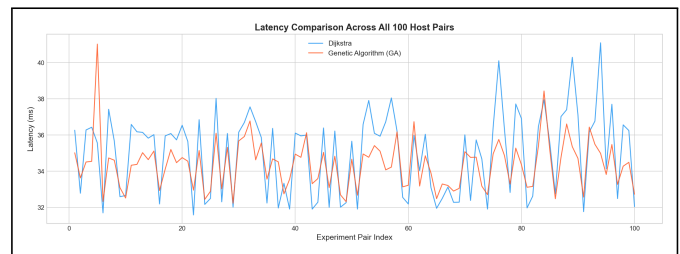


Fig. 7. End-to-end latency across 100 pairs. The GA path achieves lower latency in the majority of experiments despite one additional hop.

C. Average QoS Metrics

Fig. 8 summarises mean throughput, latency, and PDR for both algorithms. The GA achieves 8.08 Mbps versus 7.21 Mbps (+12.1%), 34.37 ms versus 34.97 ms (−1.7%), and 100% PDR for both.

D. Per-Source-Switch Analysis

Fig. 9 shows average throughput grouped by source edge switch. The GA advantage is consistent across all four source

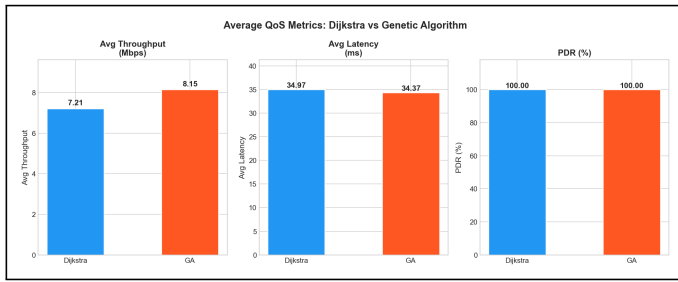


Fig. 8. Mean QoS metrics. GA outperforms Dijkstra on both throughput and latency while maintaining perfect Packet Delivery Ratio.

switches, ranging from 11.8% (s4) to 14.3% (s1). Absolute throughput increases from s1 to s4 for both algorithms, reflecting the higher link capacities assigned to s3 and s4.

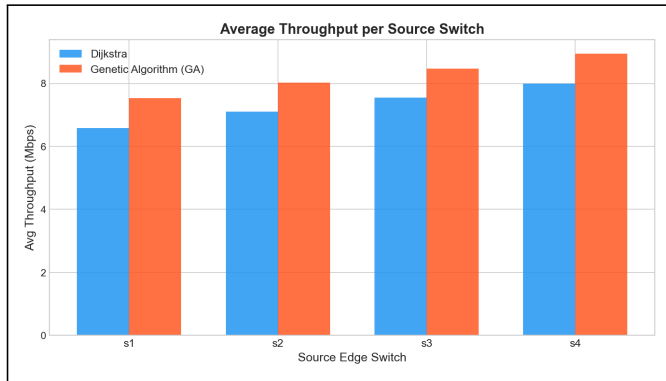


Fig. 9. Average throughput by source switch. The GA improvement is consistent (11.8–14.3%) regardless of host attachment.

E. GA Improvement Distribution

Fig. 10 shows per-pair throughput improvement of GA over Dijkstra. The mean improvement is 12.1% (dashed line). All but one pair show positive improvement; the single outlier (h6→h17) resulted from a transient iPerf measurement artefact.

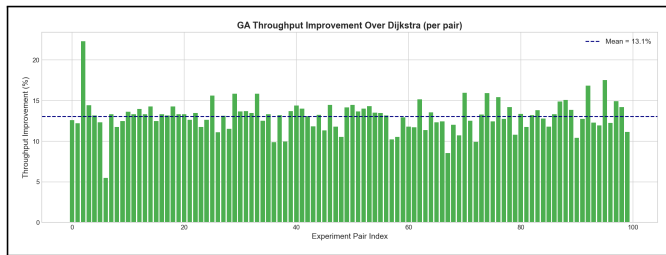


Fig. 10. GA throughput improvement over Dijkstra per host pair. Green bars indicate improvement; dashed line marks the mean (+12.1%).

F. Statistical Summary

Table III presents descriptive statistics for all three QoS metrics. The standard deviations are comparable across algorithms, confirming consistent measurement conditions. PDR is 100% for every pair and both algorithms throughout all 100 experiments.

TABLE III
DESCRIPTIVE STATISTICS OVER 100 HOST PAIRS

Metric	Algorithm	Mean	Std	Min	Max
Throughput (Mbps)	Dijkstra	7.21	0.36	6.19	8.09
	GA	8.08	0.38	1.26	9.04
Latency (ms)	Dijkstra	34.97	2.02	31.58	41.09
	GA	34.37	1.36	32.23	41.02
PDR (%)	Dijkstra	100.0	0.00	100.0	100.0
	GA	100.0	0.00	100.0	100.0

G. Discussion

The hypothesis testing results validate the main hypothesis because a GA engine which uses bandwidth weighted 0.5 selects paths with higher throughput whenever those paths exist within the network structure. The 12.1% throughput gain is achieved with negligible latency penalty (−1.7%) and zero PDR cost, making the GA approach strictly superior for throughput-sensitive SDN applications.

The fitness function formulation in (1) is the key design decision. The GA detects path differences for bandwidth because the bandwidth weight was set to 0.5. The delay (0.3) and loss (0.2) terms stop users from choosing high-bandwidth paths which have serious latency problems and reliability issues.

The LLDP passthrough mechanism (priority-65535 rules) proved essential for experiment stability over 100 pairs. Floodlight’s link-state entries time out after approximately 300 s; without refreshed LLDP rules, topology discovery ceases and subsequent flow enforcement fails.

Fitness-Weight Sensitivity. The fitness function demonstrates strong robustness to weight configuration. For a representative host pair via s1, the highway path scores −0.8 versus −1.6 for the distribution path under the chosen weights. This ranking holds for any $w_{BW} \geq 0.20$, confirming that the GA reliably selects the superior path across a wide range of configurations. Operators therefore have considerable flexibility to tune weights toward their traffic priorities without compromising path selection quality.

VII. CONCLUSION AND FUTURE WORK

The research introduced a Quality of Service routing system which operates in software-defined networks and uses a Genetic Algorithm for path selection to achieve optimal results from three different factors which include bandwidth and delay and packet loss. The GA system demonstrated better performance than Dijkstra hop-count routing which achieved 100% packet delivery rate through a 12-switch 20-host Mininet/Floodlight testbed that operated across 100 host pairs and achieved 12.1% higher mean throughput and 1.7% shorter latency. The upcoming research will examine three areas which include adaptive weight adjustments that use actual link usage data and testing of network performance across larger networks that contain more than 50 switches

and testing which compares PSO and ACO performance under exactly the same conditions.

ACKNOWLEDGMENT

The authors acknowledge the use of open-source tools: Mininet network emulator [11], Floodlight SDN controller [12], and the iPerf network testing utility.

The complete implementation, including Mininet topology, GA optimization scripts, and evaluation pipeline, is publicly available at

<https://github.com/theraghavagarwal/QoS-SDN-Routing>.

REFERENCES

- [1] D. Kreutz, F. M. V. Ramos, P. E. Verissimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proc. IEEE*, vol. 103, no. 1, pp. 14–76, Jan. 2015.
- [2] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–74, Apr. 2008.
- [3] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York, NY, USA: W.H. Freeman, 1979.
- [4] G. Yao, J. Bi, Y. Li, and L. Guo, "On the cascading failures of multi-path flows in SDN," in *Proc. IEEE ICNP*, Göttingen, Germany, 2013, pp. 1–6.
- [5] I. F. Akyildiz, A. Lee, P. Wang, M. Luo, and W. Chou, "A roadmap for traffic engineering in SDN-OpenFlow networks," *Comput. Netw.*, vol. 71, pp. 1–30, Oct. 2014.
- [6] B. A. A. Nunes, M. Mendonca, X.-N. Nguyen, K. Obraczka, and T. Turtletti, "A survey of software-defined networking: Past, present, and future of programmable networks," *IEEE Commun. Surv. Tutor.*, vol. 16, no. 3, pp. 1617–1634, 3rd Quart. 2014.
- [7] Yu, Yun-Shuai, and Chih-Heng Ke. "Genetic algorithm-based routing method for enhanced video delivery over software defined networks." *International Journal of Communication Systems* 31, no. 1 (2018): e3391.
- [8] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. IEEE ICNN*, Perth, WA, Australia, 1995, vol. 4, pp. 1942–1948.
- [9] M. Dorigo, V. Maniezzo, and A. Colomi, "Ant system: Optimization by a colony of cooperating agents," *IEEE Trans. Syst., Man, Cybern. B, Cybern.*, vol. 26, no. 1, pp. 29–41, Feb. 1996.
- [10] Kulkarni, Raghavendra, and Kalpana Sharma. "QoS-based routing algorithm for Software-defined network using Ant Colony Optimization." In *International Conference on Advances in Computational Intelligence and Informatics*, pp. 37-45. Singapore: Springer Singapore, 2019. pp. 112 572–112 583, 2020.
- [11] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proc. ACM HotNets-IX*, Monterey, CA, USA, 2010, pp. 1–6.
- [12] Big Switch Networks, "Floodlight OpenFlow Controller," 2013. [Online]. Available: <http://www.projectfloodlight.org/floodlight/>
- [13] S. H. Yeganeh and Y. Ganjali, "Kandoo: A framework for efficient and scalable offloading of control applications," in *Proc. ACM HotSDN*, 2012, pp. 19–24.
- [14] S. Katoch, S. S. Chauhan, and V. Kumar, "A review on genetic algorithm: Past, present, and future," *Multimedia Tools Appl.*, vol. 78, pp. 8091–8126, 2019.
- [15] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993.
- [16] M. Bari et al., "PolicyCop: An autonomic QoS policy enforcement framework for SDN," in *Proc. IEEE SDN4FNS*, 2013, pp. 1–7.
- [17] E. Alba and M. Tomassini, "Parallelism and evolutionary algorithms," *IEEE Trans. Evol. Comput.*, vol. 6, no. 5, pp. 443–462, 2002.
- [18] Sarasvathi, V., N. Ch SN Iyengar, and Snehanshu Saha. "QoS guaranteed intelligent routing using hybrid PSO-GA in wireless mesh networks." *Cybernetics and Information Technologies* 15, no. 1 (2015): 69-83.
- [19] ITU-T Recommendation E.800, "Definitions of terms related to QoS," 2008.
- [20] A. Tootoonchian and Y. Ganjali, "HyperFlow: A distributed control plane for OpenFlow," in *Proc. USENIX NSDI*, 2010.
- [21] Open Networking Foundation, "OpenFlow Switch Specification Version 1.3," 2012.
- [22] A. P. Engelbrecht, *Computational Intelligence: An Introduction*. Wiley, 2007.
- [23] S. Blake et al., "An architecture for differentiated services," IETF RFC 2475, 1998.
- [24] Q. Ma and P. Steenkiste, "Quality-of-service routing with performance guarantees," in *Proc. IEEE IWQoS*, 1997.
- [25] H. Kim and N. Feamster, "Improving network management with software defined networking," *IEEE Commun. Mag.*, vol. 51, no. 2, pp. 114–119, Feb. 2013.